Arrays, Strings & Exception Handling

1. Arrays in Java

An array is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed.

- **1D Arrays:** A list of variables of the same type, accessed by a common name. Example: int[] arr = new int[5];
- **2D Arrays:** Arrays of arrays, often used to represent matrices or tables. Example: int[][] matrix = new int[3][3];

2. Strings in Java

Strings, which are widely used in Java programming, are a sequence of characters. In Java programming language, strings are objects. The Java platform provides the String class to create and manipulate strings.

String immutability: String objects are immutable, which means that once created, their values cannot be changed.

Important String methods include: length(), charAt(), substring(), toLowerCase(), toUpperCase(), trim(), replace(), split().

3. StringBuilder and StringBuffer

Because String objects are immutable, manipulating them can create many temporary objects and be inefficient. StringBuilder and StringBuffer classes are used when you want to modify character strings.

- **StringBuilder:** Fast, not thread-safe.
- **StringBuffer:** Slower, but thread-safe (synchronized).

4. Exception Handling

An exception is an unwanted or unexpected event, which occurs during the execution of a program i.e at run time, that disrupts the normal flow of the program's instructions.

try-catch-finally block:

- The **try** statement allows you to define a block of code to be tested for errors while it is being executed.
- The **catch** statement allows you to define a block of code to be executed, if an error occurs in the try block.
- The **finally** statement lets you execute code, after try...catch, regardless of the result.

5. Throw, Throws and Custom Exceptions

The **throw** keyword is used to explicitly throw an exception.

The **throws** keyword is used in a method signature to declare that it might throw exceptions.

Custom Exceptions: You can create your own exception classes by extending the Exception class (for checked exceptions) or RuntimeException class (for unchecked exceptions).